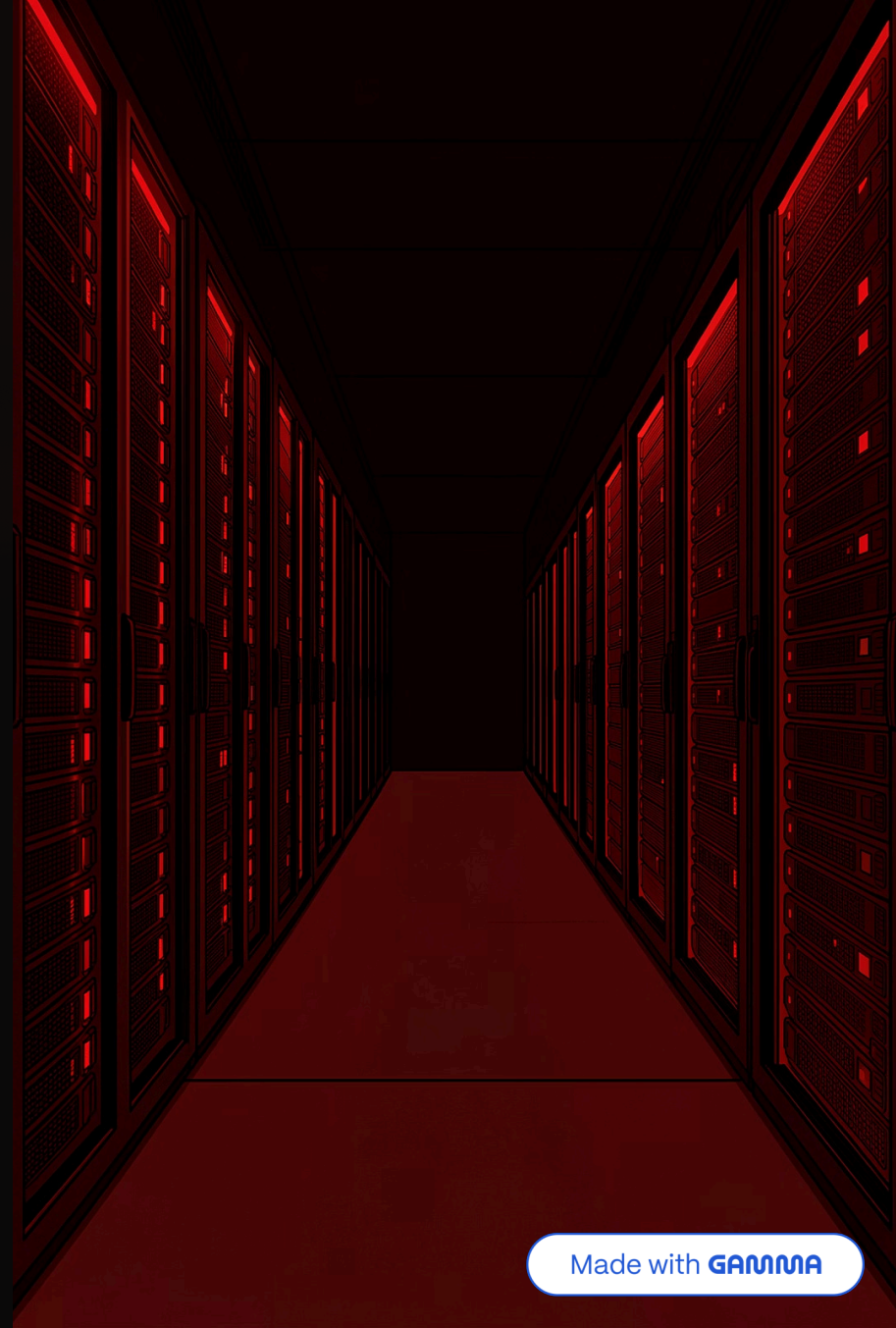
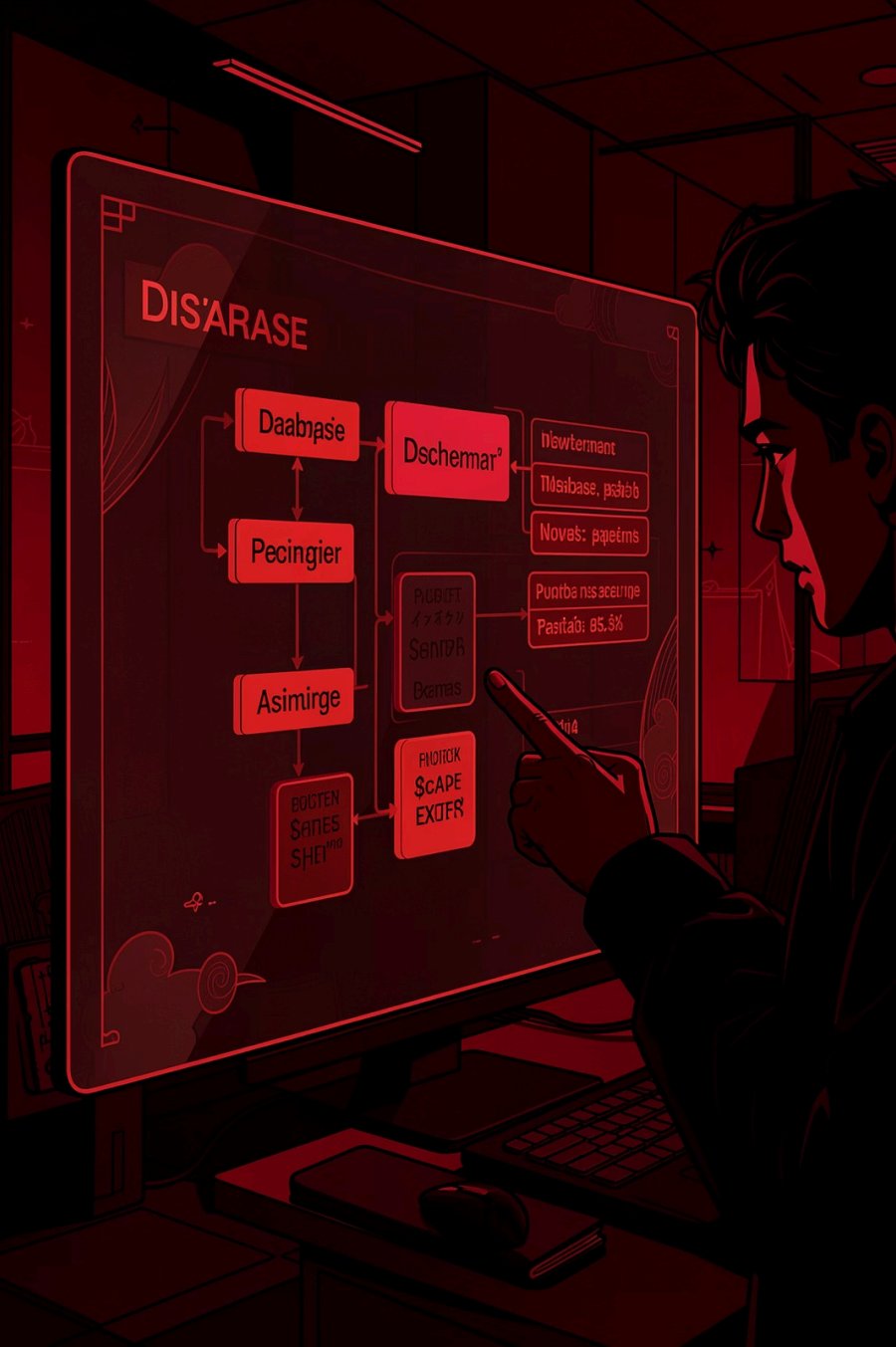


Arquitectura de Sistemas de Bases de Datos

Visión general: conceptos, modelos y decisiones técnicas para diseñar sistemas de almacenamiento de datos seguros, escalables y mantenibles.





¿Qué es la arquitectura de una base de datos?

La arquitectura de una base de datos es el diseño estructurado que define cómo se organiza, almacena y recupera la información. Su propósito clave es la **abstracción de datos**: separar la vista que tienen los usuarios de la implementación física y optimizar rendimiento, seguridad y disponibilidad.

- ❶ Diseño correcto = menor acoplamiento entre aplicaciones y almacenamiento físico.

Modelo ANSI/SPARC:

Arquitectura de tres niveles

Nivel Externo (Vistas)

Vistas personalizadas por rol (p. ej. cajero, analista). Control de acceso y presentación sin cambiar el modelo central.

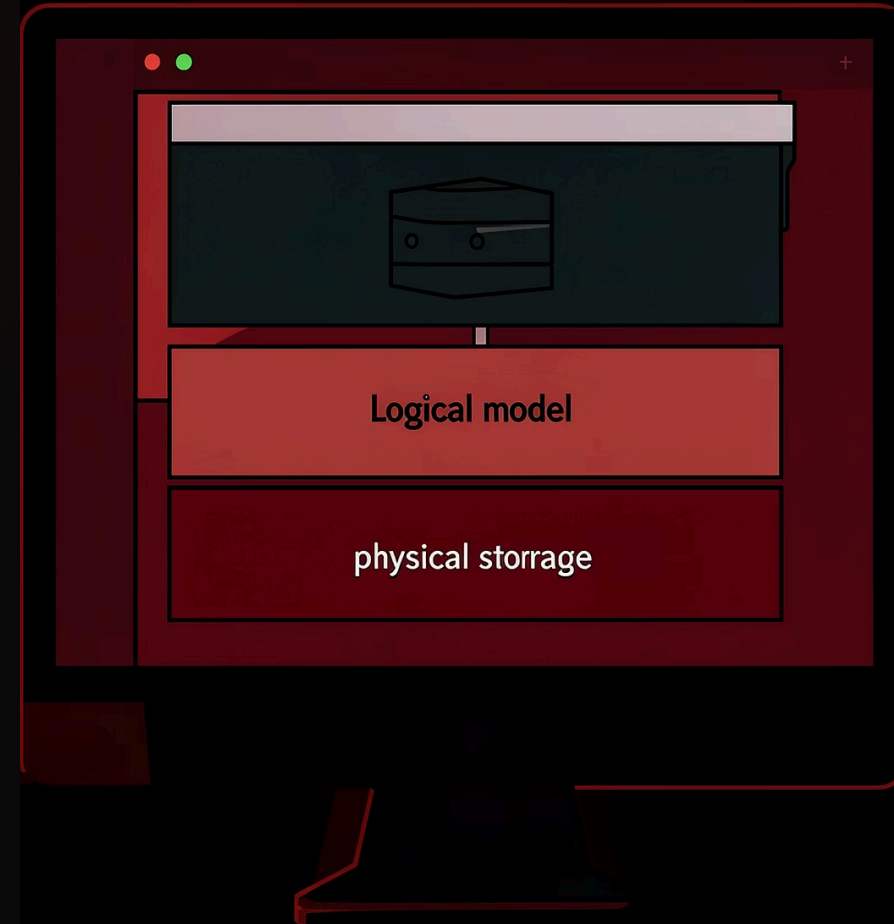
Nivel Conceptual (Lógico)

Esquema global: entidades, relaciones, atributos y restricciones. Define integridad y consistencia del dominio.

Nivel Interno (Físico)

Organización en disco, índices, formatos de almacenamiento y técnicas de compresión o particionado.

Beneficio principal: **independencia de datos** — cambios físicos sin romper vistas ni lógica.





Patrones de despliegue: cliente-servidor vs web 3 capas

Cliente-Servidor (2 capas): cliente se conecta directamente al motor. Uso: aplicaciones locales o pequeñas. Ventaja: simple; Limitación: escalabilidad y seguridad limitadas.

Web (3 capas): Presentación → Lógica de negocio → Datos. La capa intermedia (servidor de aplicaciones) centraliza reglas, cachés y control de acceso, facilitando mantenimiento y escalado.

Componentes internos del DBMS



Motor de almacenamiento

Gestiona archivos, páginas, buffers y persistencia. Afecta latencia y coste I/O.



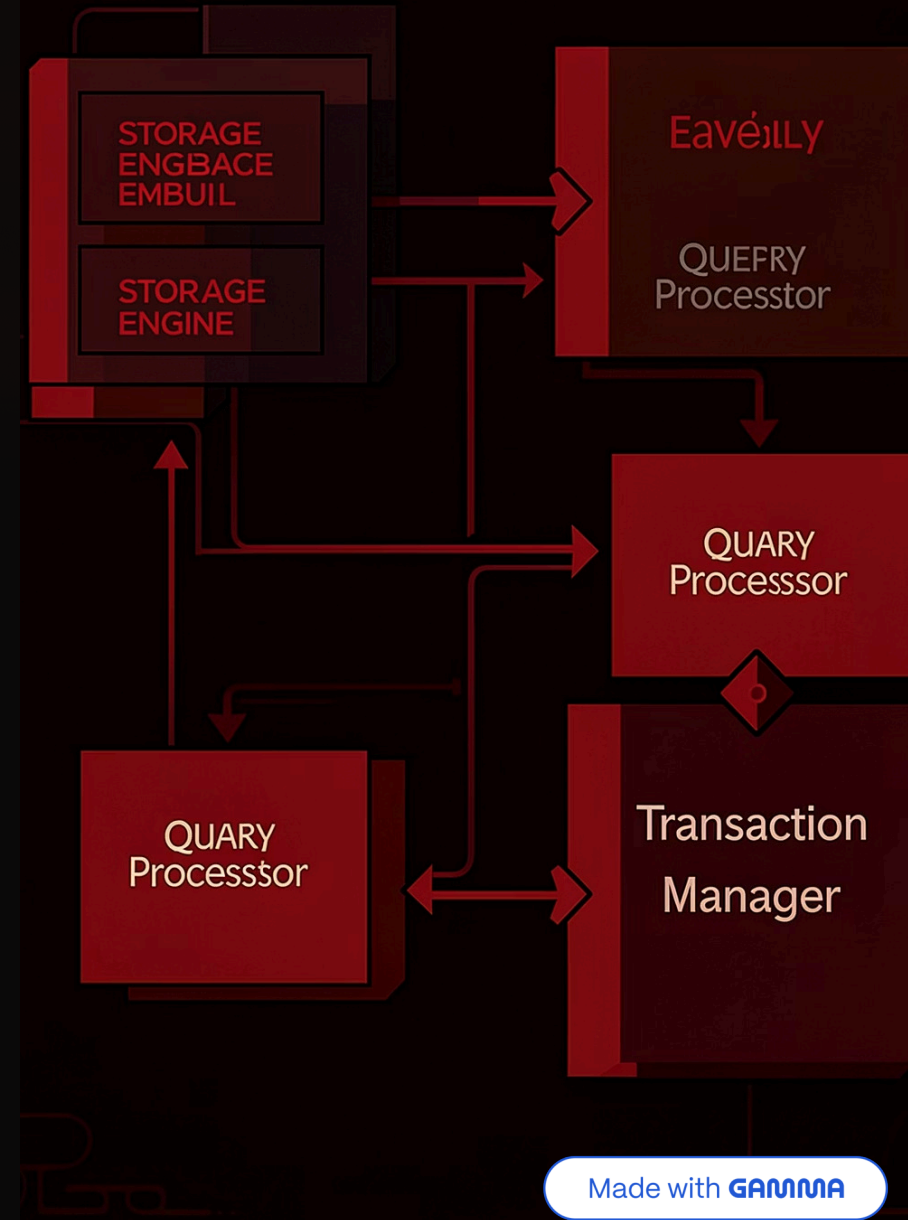
Procesador de consultas

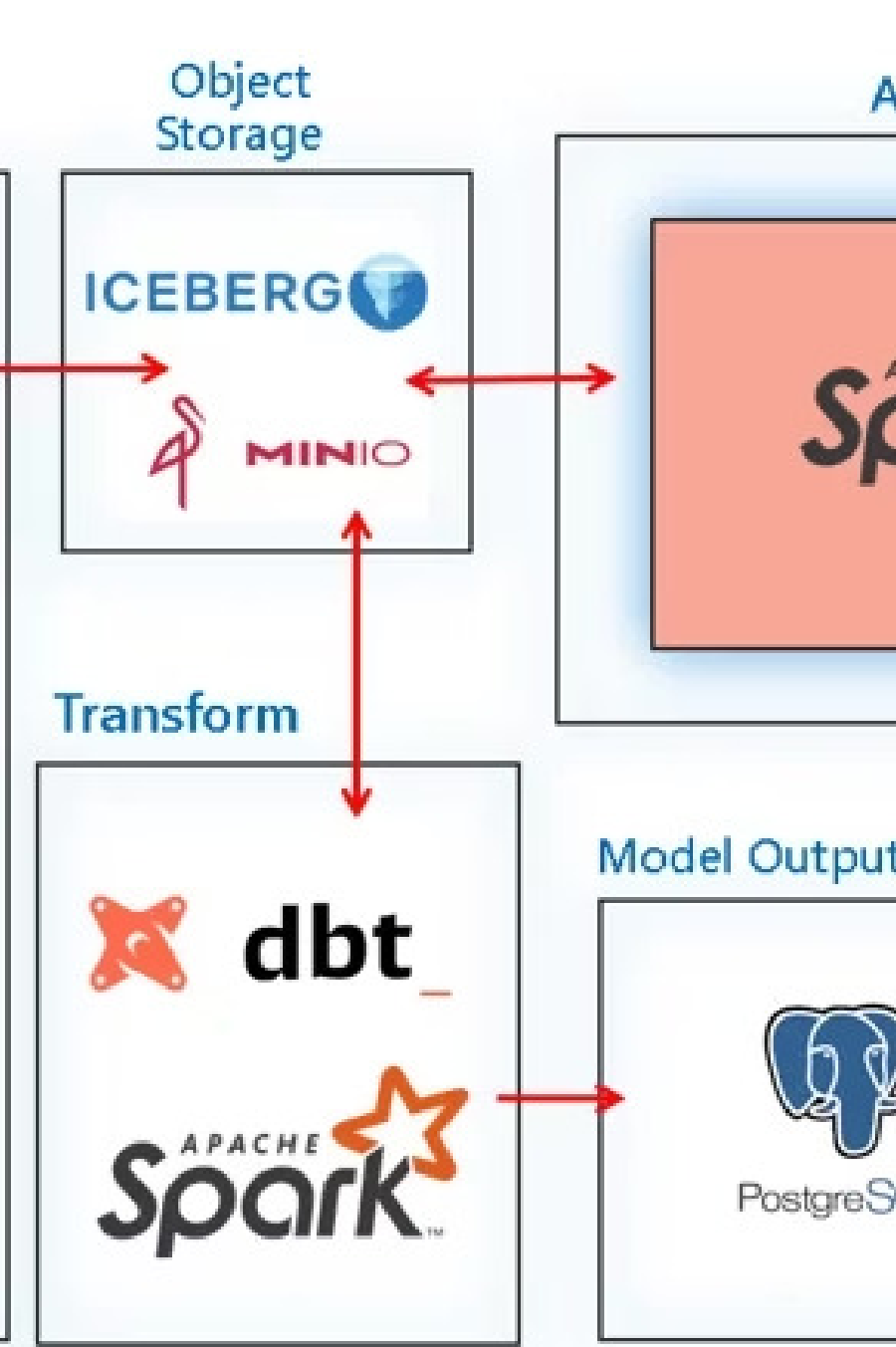
Parsers, optimizador y planificador. Decisivo para tiempos de respuesta y uso de índices.



Gestor de transacciones

Garantiza ACID: atomicidad, consistencia, aislamiento y durabilidad; controla concurrencia y recuperación.





Escalabilidad y distribución – conceptos clave

En arquitecturas de alto rendimiento aparecen patrones distribuidos para gestionar volumen, latencia y disponibilidad:

- Replicación: múltiples copias para lectura escalable y tolerancia a fallos.
- Sharding: particionado horizontal que distribuye carga y reduce latencia por fragmento.
- Bases federadas: integración de sistemas heterogéneos que exponen una vista unificada.



Diagrama: la replicación mejora lecturas y disponibilidad; el sharding mejora paralelismo en escrituras y reduce tamaño de índices por shard.

Elección práctica

1. Si predomina lectura y se requiere alta disponibilidad → replicación.
2. Si hay volumen masivo de datos y escrituras concurrentes → sharding.
3. Combinación: replicación por shard para lo mejor de ambos mundos.

Seguridad, integridad y controles

Una arquitectura robusta incorpora:

- Autenticación y autorización granulada por vistas y roles.
- Encriptación en tránsito y en reposo para protección de datos sensibles.
- Políticas de integridad (constraints, triggers) que evitan datos inconsistentes.
- Auditoría y monitoreo continuo para detectar anomalías y cumplir normativas.

⚠ No delegues validación crítica solo a la aplicación: el DBMS debe reforzar reglas de integridad.

SECURE DATABASE ACCESS CONTROL



Observabilidad y operaciones

Operar una base de datos a escala requiere: métricas (latencia, IOPS, uso de CPU/memoria), trazas de consultas lentas y alertas basadas en SLOs. Estrategias clave:

- Profiling continuo de consultas y ajuste de índices.
- Pruebas de carga y simulación de fallos (chaos engineering).
- Automatización de backup y recuperación con RTO/RPO definidos.



Conclusiones y recomendaciones

1

Prioriza independencia de datos

Mantén separación clara entre vistas, modelo lógico y almacenamiento físico para facilitar cambios sin impacto funcional.

2

Diseña según perfil de carga

Elige replicación, sharding o federación según patrones de lectura/escritura y requisitos de latencia.

3

Invierte en observabilidad y seguridad

Métricas, pruebas de rendimiento y controles de acceso son tan críticos como la elección del motor.

Próximos pasos sugeridos: auditar el modelo actual, mapear patrones de uso y prototipar particionado/replicación en un entorno controlado.

- ✓ Si lo desean, puedo generar un diagrama de migración paso a paso o una plantilla de pruebas de carga adaptada a su sistema.

